

Assignment-2

Parallel & Distributed Computing
(UCS645)

Submitted by:

Bhawesh War - 102313065

Submitted to:

Dr. Saif Nalband

Program 1: Lennard-Jones (LJ Simulation)

Execution Time (seconds)

Threads	Static	Dynamic	Guided
1	0.055361	0.024608	0.022460
2	0.042053	0.031852	0.053582
4	0.028597	0.024610	0.040513

Table 1: LJ Simulation – Execution Time

Speed Up

Threads	Static	Dynamic	Guided
1	1	1	1
2	1.32	0.77	0.42
4	1.94	1.00	0.55

Table 2: LJ Simulation – Speed Up

Efficiency

Threads	Static	Dynamic	Guided
1	1	1	1
2	0.66	0.38	0.21
4	0.49	0.25	0.14

Table 3: LJ Simulation – Efficiency

Program 2: Smith-Waterman

Execution Time (seconds)

Threads	Static	Dynamic	Guided
1	0.000028	0.000032	0.000094
2	0.000538	0.001274	0.000376
4	0.050984	0.096969	0.017397

Table 4: Smith-Waterman – Execution Time

Speed Up

Threads	Static	Dynamic	Guided
1	1	1	1
2	0.05	0.025	0.25
4	0.00055	0.00033	0.0054

Table 5: Smith-Waterman – Speed Up

Efficiency

Threads	Static	Dynamic	Guided
1	1	1	1
2	0.025	0.012	0.125
4	0.00013	0.00008	0.0013

Table 6: Smith-Waterman – Efficiency

Program 3: Heat Diffusion

Execution Time (seconds)

Threads	Static	Dynamic	Guided
1	0.107150	0.271218	0.113960
2	0.093576	0.095453	0.103222
4	0.218009	0.254751	0.245433

Table 7: Heat Diffusion – Execution Time

Speed Up

Threads	Static	Dynamic	Guided
1	1	1	1
2	1.14	2.84	1.10
4	0.49	1.06	0.46

Table 8: Heat Diffusion – Speed Up

Efficiency

Threads	Static	Dynamic	Guided
1	1	1	1
2	0.57	1.42(measurement anomaly)	0.55
4	0.12	0.26	0.11

Table 9: Heat Diffusion – Efficiency

Observations

1. Smith-Waterman

- Extremely small input $\Rightarrow$ execution time is too small.
- Parallel overhead dominates.

2. LJ Simulation

- Best scaling with static scheduling.
- Dynamic/guided overhead clearly visible.

3. Heat Diffusion

- Best performance at 2 threads.
- Performance drops at 4 threads due to hyperthreading overhead.

Perf Analysis

1. Lennard-Jones (LJ Simulation)

Metric	1 Thread (Baseline)	4 Threads (Static + Best)
Execution Time (s)	0.01807	0.01331
Cycles	53,719,037	114,907,526
Instructions	60,285,872	63,576,446
IPC	1.12	0.55
Task-clock	17,403,161	37,440,651
Context Switches	0	32
Branch Miss Rate(%)	0.98	0.60

Table 10: LJ Simulation – Perf Metrics

Observations:

- **Performance Improvement:** Execution time reduced from 0.018 s $\rightarrow$ 0.013 s, showing parallel speedup (though not ideal).
- **IPC Drop (Important):** IPC drops from 1.12 $\rightarrow$ 0.55, meaning the CPU performs less useful work per cycle. Reasons: thread contention, memory access delays, synchronisation overhead.
- **Cycles Increased:** Cycles almost doubled due to more threads running simultaneously and increased overhead.
- **Context Switching:** Rises from 0 $\rightarrow$ 32, indicating OS scheduling overhead.

2. Smith-Waterman

Metric	1 Thread (Baseline + Best)
Execution Time (s)	0.00120
Cycles	1,974,206
Instructions	1,253,467
IPC	0.63
Task-clock	645,518
Context Switches	0
Branch Miss Rate(%)	4.66

Table 11: Smith-Waterman – Perf Metrics

Observations:

- **Very Low Execution Time:** Extremely small runtime at the microseconds level.
- **Low IPC (0.63):** Indicates inefficient pipeline usage due to dependency-heavy computation.
- **High Branch Miss Rate (4.66%):** Much higher than other programs. Reason: dynamic programming involves many conditional checks and branch-heavy logic.

- **No Benefit from Parallelism:** Already evident in execution time tables. Cause: small input size causes parallel overhead to dominate.

3. Heat Diffusion

Metric	1 Thread (Static)	2 Threads (Dynamic + Best)
Execution Time (s)	0.07917	0.04559
Cycles	238,827,889	267,432,657
Instructions	797,513,926	802,441,568
IPC	3.34	3.00
Task-clock	77,402,118	88,742,180
Context Switches	23	8
Branch Miss Rate(%)	0.63	0.68

Table 12: Heat Diffusion – Perf Metrics

Observations:

- **Significant Speedup:** Execution time reduced from 0.079 s $\rightarrow$ 0.045 s. Good parallel scaling.
- **High IPC (Very Important):** IPC $\approx$ 3.0–3.34, indicating highly efficient CPU utilisation, compute-heavy workload, and strong instruction-level parallelism.
- **Instructions $\approx$ Same:** The same total work is performed, just executed faster.
- **Slight IPC Drop:** 3.34 $\rightarrow$ 3.00, indicating thread overhead and memory contention.
- **Reduced Context Switching:** Drops from 23 $\rightarrow$ 8, indicating better scheduling with dynamic policy.

LIKWID Analysis

1. Lennard-Jones (LJ Simulation)

Metric	FLOPS_DP	MEM	L2CACHE
Execution Time (s)	0.035513	0.038323	0.059422

Table 13: LJ Simulation – LIKWID Groups

Observations:

- **High Computational Workload:** LJ involves `sqrt()`, `pow()` – heavy floating-point operations. The FLOPS group shows strong compute activity.
- **Memory Involvement:** MEM time slightly higher than FLOPS, indicating frequent data access since particle interactions require memory reads.
- **Cache Impact:** L2CACHE time highest (0.059 s), indicating cache misses / pressure and sub-optimal data reuse.

2. Smith-Waterman

Metric	FLOPS_DP	MEM
Execution Time (s)	0.000027	0.000030

Table 14: Smith-Waterman – LIKWID Groups

Observations:

- **Extremely Low Runtime:** Execution time at the microseconds level, too small for meaningful hardware analysis.
- **Negligible FLOPS:** Very few floating-point operations; the workload is integer-heavy and comparison-based.
- **Minimal Memory Bandwidth:** Small dataset fits entirely in cache.

3. Heat Diffusion

Metric	FLOPS_DP	MEM	L2CACHE
Execution Time (s)	0.045722	0.046427	0.057429

Table 15: Heat Diffusion – LIKWID Groups

Observations:

- **Balanced Compute + Memory Workload:** FLOPS and MEM times are very close, indicating that both computation and memory are equally important.
- **Memory Bandwidth Influence:** MEM slightly higher than FLOPS, making memory access a limiting factor.
- **Cache Behaviour:** L2CACHE time highest, indicating repeated grid access and cache pressure.

- **Stable Execution:** All three metrics close together, confirming a balanced workload.